

LFX Mentorship Proposal

Automated Documentation Sync Bot for krkn-chaos Projects

CNCF - krkn-chaos | LFX Term 2, 2026

Dibyajyoti Mandal

<https://github.com/dibyajyoti-mandal>

Project	<u>krkn-chaos/website #320</u>
LFX URL	<u>mentorship.lfx.linuxfoundation.org/project/5b44929d-7570-43a7-80ea-474062b813fb</u>
Mentors	Rahul Shetty · Naga Ravi Chaitanya Elluri · Darshan Jain
Stack	Python · GitHub Actions · LLM API · Hugo / Docsy

1. Introduction

How did you learn about our mentoring program?

I learned about the LFX Mentorship program from a post on LinkedIn. Upon researching more about the program and the projects involved, I found this specific project under open applications.

Why are you interested in this program?

I have been deeply interested in software development since childhood and have built substantial experience across the field over the years. This project under LFX mentorship aligns directly with my interests in automation infrastructure, developer tooling, and the intersection of AI-assisted workflows. Working with a CNCF project like **krkn-chaos**, will push me to think carefully about correctness, maintainability, and the developer experience of the contributors who depend on these tools every day. I am genuinely excited by the opportunity to build something that becomes a durable part of the project's infrastructure rather than a one-off contribution, and I look forward to contributing.

About Me

I am Dibyajyoti Mandal, currently an undergraduate student pursuing a B.Tech in Electronics Engineering at the **Indian Institute of Technology (BHU) Varanasi**, where I maintain a grade of **9.46**. My domain of interest lies in scalable software architecture and AI-driven systems, making me uniquely suited for the project. Some of my significant projects include :

- During my tenure as a **Software Engineering Intern** at **Salesforce**, I developed a C#.NET library that replicated a Golang configuration microservice, which reduced the turnaround time by approximately 85%. This experience heavily trained me in building highly efficient, production-grade microservices and navigating complex architectural dependencies.
- At **Qualcomm's** Edge AI Developer hackathon we built a context injection tool [ContextWeaver](#) for LLMs leveraging the Neural Processing Unit for running inference on **FastAPI** based backends.
- I have built [GoJudge](#), a secure, remote code execution engine using Go, Redis Streams as a job queue, and Docker for containerized execution environments. This was directly inspired by the [judge0](#) architecture.

My research under Dr. Om Jee Pandey at IIT BHU involved implementing novel federated learning architectures using algorithms like Shamir's Secret Sharing, achieving up to a 70% improvement in model accuracy (publication currently under IEEE review). Lastly, I have also had some experience in open source contributions, specifically in the **scancode-toolkit** and **metabrainz** projects.

I am deeply interested in building robust cloud infrastructure for advanced AI, particularly involving Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and model quantization. My primary strengths include:

- **Backend & Systems Engineering:** Highly proficient in Python, Go, C++, and JavaScript/TypeScript, with strong hands-on experience using frameworks like FastAPI, Fiber and containerization tools like Docker.
- **AI Infrastructure & LLMops:** Hands-on experience deploying and optimizing local and cloud-based models using **Ollama** and **vLLM**. Skilled in building complex **RAG** pipelines and autonomous agents using **LangChain** and **LlamaIndex**.
- **Database & Data Management:** Experienced in structuring and querying data using SQL, Redis, MongoDB, and CouchDB.

What experience and knowledge/skills do you have that are applicable to this program?

- **CI/CD and event-driven automation** : I have hands-on experience designing and implementing pipeline workflows, which maps directly to the GitHub Actions architecture at the heart of this bot, including reusable workflows, artifact passing between jobs, and webhook-triggered event handling.
- **Python systems design** : My experience with Python extends to building multi-component systems with clean separation of concerns, which is exactly the discipline needed for a pipeline where the trigger, classifier, generator, and PR creator must each be independently testable and maintainable.
- **LLM API integration** : I have worked with large language model APIs in practical contexts, including prompt engineering for structured JSON outputs, few-shot example design, and building retry-resilient client wrappers - all of which are critical for the doc generation stage where consistency and graceful failure handling matter as much as output quality.
- **Static site familiarity** : I have working knowledge of static site generators and their content conventions, giving me a head start in understanding the Hugo/Docsy structure at krkn-chaos.dev well enough to generate content that fits the site rather than breaking it.
 - Github : <https://github.com/dibyajyoti-mandal>
 - Email : dibyajyoti.mandal.cd.ece22@itbhu.ac.in / dmandalismyname@gmail.com
 - Contact no. : +91 9547710746
 - Resume : [dibyajyoti_resume](#)
 - Time Zone : **Indian Standard Time (UTC + 5:30)**

2. The Problem

Background

The krkn-chaos ecosystem is a suite of Kubernetes chaos-engineering tools maintained under the CNCF umbrella. It spans multiple repositories: **krkn** (core engine), **krkn-hub** (scenario library), **krknctl** (CLI), **krkn-ai** (AI-assisted runs), **krkn-operator** (Kubernetes operator), and **cerberus** (health monitoring). All of these are documented centrally at krkn-chaos.dev, a Hugo/Docsy static site hosted in the krkn-chaos/website repository.

The Documentation Drift Problem

Today, every time an upstream repository merges a change that affects user-facing behaviour - a new chaos scenario, a renamed config field, a new CLI flag, a modified parameter table - someone must manually open a separate pull request against the website repository to keep the docs in sync. This process is entirely manual, undocumented in any automated sense, and critically dependent on individual contributors remembering to do it.

Core pain point

Documentation drift is not an edge case - it is the default outcome. Without automation, the docs at krkn-chaos.dev inevitably lag behind the tools they describe, eroding user trust and increasing support burden.

Specific failure modes observed

- A new chaos scenario is merged into krkn-hub. No corresponding docs page is created. Users discover the scenario only by reading source code.
- A config field is renamed in krkn. The docs still show the old name. Users spend hours debugging deployment failures caused by stale documentation.
- krknctl adds a new flag. The CLI reference page is not updated. Operators cannot discover the capability without reading the commit log.
- A parameter's default value changes. The parameter table in the docs reflects the old default. Silent correctness bugs follow.

Why this is hard to solve manually

The krkn-chaos ecosystem is actively developed across six or more repositories simultaneously. No single person has a complete view of every change landing across all repos. Code reviewers are focused on correctness and performance, not documentation coverage. The website repo lives separately, so there is no natural forcing function that connects a merged upstream PR to a docs update. The result is structural: the incentive and the responsibility are misaligned, so documentation inevitably falls behind.

Scale of the problem

As krkn-chaos grows - more scenarios, more configuration surfaces, more CLI commands - the documentation gap widens faster. The only sustainable fix is automation that treats documentation as a first-class output of every upstream change.

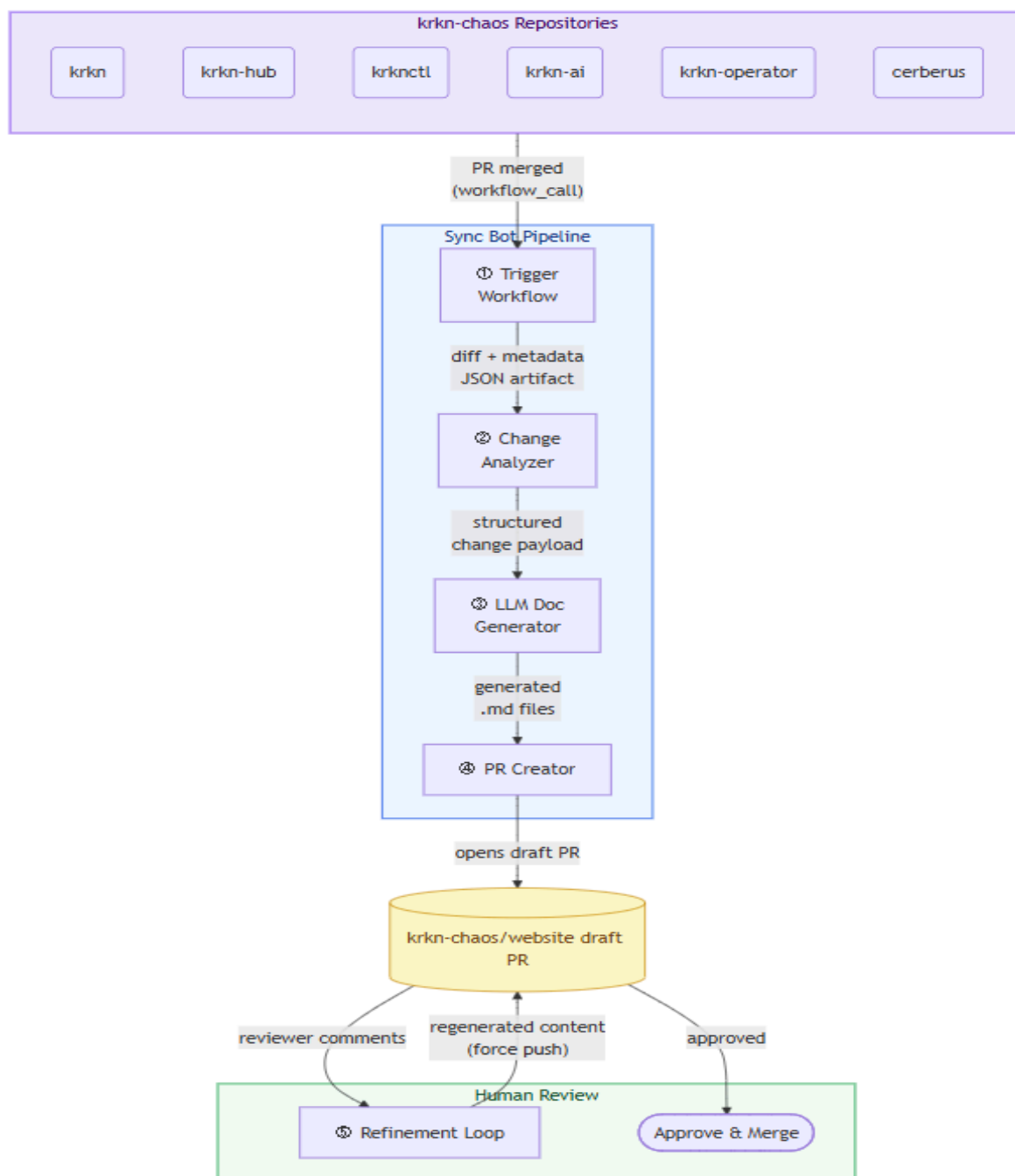
3. Solution Approach

Overview

The proposed solution is a GitHub Actions-based bot written in Python that monitors all upstream krkn-chaos repositories for merged PRs, classifies the documentation impact of each change, and automatically generates a draft pull request on the krkn-chaos/website repository with updated Hugo/Docsy content. An interactive refinement loop allows reviewers to request specific changes directly via PR comments, and the bot responds by regenerating the relevant sections.

Design philosophy

The bot does not replace human review, rather it eliminates the blank-page problem. Every draft PR it creates is a starting point that a human approves, refines, and merges. The bot handles discovery, classification, and first-draft generation; humans handle accuracy, tone, and final sign-off.



Architecture : five-stage pipeline

The system is structured as a linear pipeline with a feedback loop at the end. Each stage has a single responsibility and a well-defined interface to the next.

#	Stage	Responsibility
1	Trigger	GitHub Actions workflow_call fires on merged PR in any upstream repo and extracts diff + metadata
2	Change Analyzer	Classifies the diff into change types (new scenario / config / CLI flag) and builds a structured JSON payload
3	LLM Doc Generator	Consumes the payload and existing page content; prompts an LLM to generate or update Hugo/Docsy markdown
4	PR Creator	Writes generated .md files to a branch on krkn-chaos/website and opens a draft pull request via GitHub API
5	Refinement Loop	Listens for review comments on the draft PR; parses intent and re-invokes the LLM to push updated content

The architecture diagram shown above visualizes this design.

Stage 1 : Trigger workflow

A reusable GitHub Actions workflow (workflow_call) is installed in each upstream repository. When a PR is merged (pull_request_target event, types: [closed], with merged == true), the workflow collects: the repository name, the PR number and title, the full unified diff of changed files, and the list of file paths touched. This metadata is passed as a JSON artifact to the central dispatcher running in a dedicated bot repository.

Key design decision

Using **workflow_call** with a centralized dispatcher means the bot logic lives in one place. Adding a new upstream repo requires only adding one 3-line caller workflow to that repo - no duplication of bot code.

```
# .github/workflows/docs-sync-caller.yml
name: Docs Sync

on:
  pull_request_target:
    types: [closed]

jobs:
  trigger:
    if: github.event.pull_request.merged == true
    uses: krkn-chaos/docs-sync-bot/.github/workflows/dispatcher.yml@main
    with:
```

```
repo_name: ${ github.repository }}
pr_number: ${ github.event.pull_request.number }}
pr_title:  ${ github.event.pull_request.title }}
secrets:
  BOT_TOKEN: ${ secrets.DOCS_SYNC_BOT_TOKEN }}
```

Stage 2 : Change analyzer

This is the most critical and complex component. The analyzer reads the unified diff and applies a set of heuristics to determine whether documentation is impacted and what kind of update is needed. It outputs a structured JSON payload that downstream stages consume.

Change type classification

- **New chaos scenario** - detected by the appearance of a new YAML file under scenarios/ or a new Python module following scenario naming conventions. The analyzer extracts the scenario name, description, input parameters, and example invocation from the file.
- **Config field changes** - detected by diffing JSON Schema, Pydantic model, or dataclass definitions. The analyzer identifies added, removed, or renamed fields along with their types and default values.
- **CLI flag changes** - detected by diffing argparse add_argument calls (Python) or cobra Command definitions (Go). Extracts flag names, short forms, types, defaults, and help strings.
- **README / inline doc changes** - detected when .md files inside upstream repos are modified. The relevant paragraphs are extracted and flagged for inclusion in the site.

False positive filtering

Not every merged PR warrants a documentation update. The analyzer applies a filter pass before classification: changes touching only test files, CI configuration, internal utility functions, or vendored dependencies are silently skipped. This prevents the bot from opening spurious PRs that add noise without value.

Content to page mapping

A mapping registry (a YAML config file committed to the bot repository) records how upstream file paths map to website content paths. For example, **krkn-hub/scenarios/pod_disruption/** maps to **content/scenarios/pod-disruption/_index.md**. This registry is maintained by the mentors and contributors; the bot reads it at runtime rather than hardcoding assumptions.

Extensibility

New repos and new file-path mappings can be registered without touching the bot's Python code - only the YAML registry needs updating. This makes the system maintainable as the krkn-chaos ecosystem grows.

Changes classification has to be deterministic and cheap - regex + AST parsing is faster, cheaper, and more auditable than asking an LLM whether a diff contains a new scenario. The LLM is reserved for the expensive creative step (generation), not the mechanical one (classification).

```

# analyzer/main.py
import argparse, json
from pathlib import Path
from classifiers import ScenarioClassifier, ConfigClassifier, CLIClassifier,
ReadmeClassifier
from registry import MappingRegistry
from filters import is_noise

def analyze(repo: str, pr: int, title: str, diff: str) -> dict | None:
    if is_noise(diff):          # skip test-only / CI-only / vendor-only diffs
        return None

    registry = MappingRegistry.load("mappings.yaml")
    classifiers = [ScenarioClassifier(), ConfigClassifier(), CLIClassifier(),
ReadmeClassifier()]

    changes = []
    for clf in classifiers:
        result = clf.classify(diff, repo)
        if result:
            result["target_page"] = registry.resolve(repo, result["source_path"])
            changes.append(result)

    if not changes:
        return None

    return {"repo": repo, "pr": pr, "title": title, "changes": changes}

```

Each of the classifier classes has to be implemented separately.

Stage 3 : LLM doc generator

Once the change analyzer produces a structured payload, the LLM doc generator is responsible for turning that payload into actual Hugo/Docsy markdown. It is designed around three prompt components that work together: a system prompt encoding site conventions, a context block providing the existing page content, and a task block describing the specific change.

Prompt engineering strategy

- **System prompt** - encodes the Hugo/Docsy conventions used on krkn-chaos.dev: front matter structure, tabbed scenario layout shortcodes, parameter table format (| Field | Type | Default | Description |), and the expected heading hierarchy.
- **Context block** - the current content of the target page is injected so the LLM understands what already exists and produces a minimal, surgical update rather than a full rewrite.
- **Task block** - the structured JSON payload from the change analyzer, describing exactly what changed and where it maps on the page.
- **Few-shot examples** - two or three real examples of existing krkn-chaos docs pages are included in the system prompt to anchor the output style and format.

Output format

The LLM is instructed to return a structured **JSON response** containing: the full updated markdown content for the target page, a one-paragraph human-readable summary of what changed (used as the PR description), and a list of the specific line ranges that were modified (used to generate a focused review diff). JSON output is parsed and validated before any file write is attempted.

Stage 4 : Draft PR creator

The PR creator uses the GitHub REST API (via PyGitHub) to create a branch on krkn-chaos/website, write the generated .md files, and open a draft pull request. The PR body includes: a link back to the upstream PR that triggered the update, the LLM-generated change summary, a checklist of items for the human reviewer, and a usage guide for the interactive refinement commands.

Draft PRs by design

All created PRs are opened as drafts. This signals clearly to reviewers that automated content requires human validation before merging. The bot never opens a ready-for-review PR autonomously.

Idempotency is enforced: before opening a new PR, the bot checks whether a draft PR already exists for the same upstream PR number. If one exists, it updates the branch instead of creating a duplicate.

```
# pr_creator/creator.py
from github import Github

class PRCreator:
    LABEL = "auto-docs"

    def __init__(self, token: str):
        self.gh = Github(token)
        self.website = self.gh.get_repo("krkn-chaos/website")

    def run(self, payload: dict, md_files: dict[str, str], summary: str):
        upstream_pr_ref = f"{payload['repo']}#{payload['pr']}"
        branch_name =
f"docs/sync/{payload['repo'].split('/')[-1]}-pr-{payload['pr']}"

        # idempotency check
        existing = self._find_existing_pr(upstream_pr_ref)
        if existing:
            self._update_branch(existing, md_files)
            return existing.html_url

        # fresh draft PR
        self._create_branch(branch_name)
        self._write_files(branch_name, md_files)
```

```

        pr = self.website.create_pull(
            title = f"docs(auto): {payload['title']}",
            body = self._build_pr_body(payload, summary),
            head = branch_name,
            base = "main",
            draft = True,
        )
        pr.add_to_labels(self.LABEL)
        pr.create_issue_comment(WELCOME_COMMENT)
        return pr.html_url

    def _find_existing_pr(self, upstream_ref: str):
        for pr in self.website.get_pulls(state="open", labels=[self.LABEL]):
            if upstream_ref in pr.body:
                return pr
        return None

    def _write_files(self, branch: str, files: dict[str, str]):
        for path, content in files.items():
            try:
                existing = self.website.get_contents(path, ref=branch)
                self.website.update_file(path, f"docs(auto): update {path}",
                                         content, existing.sha, branch=branch)
            except Exception:
                self.website.create_file(path, f"docs(auto): add {path}",
                                         content, branch=branch)

```

Stage 5 : Interactive refinement loop

After a draft PR is opened, reviewers can request specific changes by leaving comments addressed to the bot. The refinement loop listens for `issue_comment` events on the draft PR, parses the reviewer's intent, re-invokes the LLM with the clarification as an additional instruction, and pushes the updated content as a new commit on the existing branch.

Supported refinement commands

- `@krkn-docs-bot expand [section]` - regenerate the named section with more detail
- `@krkn-docs-bot fix [description]` - apply a specific correction to the generated content
- `@krkn-docs-bot regenerate` - discard the current draft and regenerate from scratch
- `@krkn-docs-bot add example` - append a usage example to the relevant scenario or config section

The bot replies to each comment with a status message (processing, done, or error with reason) so reviewers have clear feedback. All bot actions are logged in the PR timeline, creating an auditable history of what was generated, when, and in response to what instruction.

4. Testing Plan

Testing strategy overview

The testing plan covers four layers: unit tests for individual components, integration tests for the end-to-end pipeline, regression tests using real historical PRs from upstream repos, and manual acceptance testing on a staging fork of the website. Each layer targets a different class of failure and together they provide high confidence in correctness before any output touches the production website.

Unit testing - change analyzer

The change analyzer is the highest-risk component because an incorrect classification causes either a missed documentation update or a spurious PR. Unit tests cover every classification heuristic in isolation.

Test cases

- **Scenario detection** - provide synthetic diffs that add a new YAML scenario file and assert that the output payload correctly identifies it as a new-scenario change with the right field extractions.
- **Config field detection** - provide diffs that add, remove, and rename Pydantic model fields; assert correct field-level diff in the payload.
- **CLI flag detection** - provide diffs to argparse and cobra definitions; assert correct flag metadata extraction.
- **False-positive filtering** - provide diffs that touch only test files, CI YAML, and vendor directories; assert the analyzer returns no-op (skip).
- **Edge cases** - empty diffs, binary file changes, very large diffs, PRs with changes across multiple change types simultaneously.

Unit testing - LLM doc generator

LLM integration is tested via mocking to avoid cost and non-determinism in CI. The tests validate prompt construction, response parsing, and error handling rather than LLM output quality (which is covered by manual review).

- Prompt builder - assert that the system prompt, context block, and task block are assembled correctly for each change type.
- Response parser - given a set of well-formed and malformed LLM JSON responses, assert correct extraction or graceful error handling.
- Retry logic - assert that transient API errors trigger retries with exponential backoff and that persistent failures surface a clear error without crashing the workflow.

Integration testing - End to end pipeline

Integration tests run the full pipeline against a staging fork of krkn-chaos/website using a GitHub App with limited scope. They use real (but historical) upstream PR diffs sourced from public commit history.

- **Trigger to payload** - fire a synthetic workflow_call event and assert the dispatcher receives a well-formed metadata artifact.
- **Payload to draft PR** - given a known payload, assert that a draft PR is opened on the staging fork with the expected branch name, file paths, and PR body structure.
- **Idempotency** - run the pipeline twice for the same upstream PR; assert that only one draft PR exists on the staging fork after both runs.
- **Refinement commands** - post a bot-addressed comment on the staging draft PR and assert that a new commit appears on the branch within the expected time window.

Regression testing - historical PRs

A curated set of 10-15 real historical upstream PRs that did result in manual doc updates are used as ground-truth regression tests. The bot's output for each is compared against the human-written doc update to measure coverage and quality.

Quality metric

Target: the bot's generated content covers at least 90% of the factual updates present in the human-written doc, with zero hallucinated parameter names or values. Quality is assessed by the mentors during the mid-term review.

Manual acceptance testing

Before any production deployment, the full pipeline is run against the actual krkn-chaos/website staging branch with sign-off from all three mentors. Acceptance criteria:

1. Draft PRs open correctly and contain no broken Hugo shortcodes or invalid front matter.
2. The PR body links correctly to the upstream PR and the change summary is accurate.
3. At least three refinement commands are tested and produce correct incremental updates.
4. No duplicate PRs are created across multiple test runs for the same upstream PR.
5. The bot does not open any PRs for upstream changes that do not affect user-facing documentation.

5. Timeline

Here's a rough 12-week timeline:

Weeks 1–2 : Onboarding and deep dive Get familiar with the full krkn-chaos ecosystem, study the website repository's Hugo/Docsy content structure and conventions in detail, audit 10–15 historical upstream PRs that required manual doc updates to use as ground-truth regression cases, and align with mentors on scope, tooling choices, and the initial mapping registry structure.

Weeks 3–4 : Trigger workflow and change analyzer (foundation) Build and test the reusable GitHub Actions caller workflow and central dispatcher. Implement the false-positive filter and the four change-type classifiers (scenario, config, CLI flag, README). Build the YAML mapping registry. Unit test every classifier against synthetic and real historical diffs.

Weeks 5–6 : LLM doc generator Design the prompt architecture (system, context, task blocks) with few-shot examples drawn from real krkn-chaos.dev pages. Implement the LLM API client with retry and backoff logic. Build the JSON response parser and schema validator. Test output quality against the historical PR regression set and iterate on prompts.

Weeks 7–8 : PR creator and end-to-end integration Build the draft PR creator with idempotency checking, branch management, PR body assembly, and label application. Wire all stages together into the full end-to-end pipeline. Run integration tests on a staging fork of the website. Fix any interface mismatches between stages surfaced by integration testing.

Weeks 9–10 : Refinement loop and interactive bot Implement the `issue_comment` webhook listener, intent parser, and the four refinement commands (`expand`, `fix`, `add example`, `regenerate`). Build the acknowledgment-and-edit comment pattern for reviewer feedback. Test the full review cycle end-to-end on the staging fork with mentor participation.

Weeks 11–12 : Hardening, documentation, and handoff Run the full regression test suite, fix any remaining edge cases, and conduct manual acceptance testing with all three mentors. Write developer documentation for extending the mapping registry and adding new classifiers. Prepare a final demo and handoff notes covering architecture decisions, known limitations, and suggested next steps for post-term extensions.

Should any unforeseen spillovers arise, I am fully prepared to dedicate the necessary time post the mentorship timeline to ensure every aspect of the project is successfully implemented

6. Conclusion

What this project establishes

The Term 2 deliverable is intentionally scoped: a working bot that handles the four most common change types (new scenario, config changes, CLI flag changes, README propagation) across the six primary krkn-chaos repositories. This creates a foundation that several valuable extensions can build on.

Short-term extensions (post-Term 2)

Multi-language support

The current change analyzer handles Python (argparse, Pydantic) and Go (cobra) CLI definitions. Extending it to parse YAML-based configuration schemas used by the krkn-operator and Helm chart values.yaml files would significantly expand coverage without requiring LLM involvement - these are structured formats amenable to deterministic parsing.

Automated screenshot generation

Several krkn-chaos docs pages include terminal output examples. A future extension could run `krkntcl` in a sandboxed container after a CLI change and capture updated terminal output screenshots or code blocks automatically, keeping examples current without any manual effort.

Documentation coverage dashboard

A lightweight GitHub Pages dashboard that tracks, per repository, which scenarios and config surfaces have up-to-date documentation and which are stale. This makes the documentation gap visible at a glance and creates accountability for human contributors.

Long-term vision

The ultimate goal is zero documentation drift: a state where the docs at `krkn-chaos.dev` are always accurate because every code change that affects user-visible behaviour automatically produces a corresponding documentation update that a human simply reviews and approves. This project is the first step toward that goal - replacing the blank-page problem with a review problem, which is a much better position to be in.

Broader impact

The architecture designed here - trigger workflow + structured change analyzer + LLM content generator + interactive refinement - is generic enough to be extracted as a reusable GitHub Action that other CNCF projects facing the same documentation drift problem could adopt. That reuse potential makes this project multiply valuable.

What do you hope to get out of this mentorship experience?

By the end of this mentorship I hope to have a deeper understanding of how production-grade automation systems are designed, built, tested, and shipped - the full lifecycle from an initial architectural decision through to something real contributors depend on every day.

Beyond the technical depth, I am looking forward to experiencing what it means to contribute meaningfully to a CNCF ecosystem project, understanding how decisions get made in the open, how to work with maintainers, and what separates a good open source contribution from a great one. I strongly believe this experience will shape how I approach complex software problems well into the future. I would also love to stay connected with the krkn-chaos community and keep contributing to the project after the mentorship term ends, whether by extending the bot's capabilities or helping onboard future contributors to the codebase.